BCO701 IoT Lab — Short Programs

The same 12 experiments, cut down to the fewest lines that still do the job. Each one has the connections, a one-line memory hook, the program and the expected output. All programs were compile-checked.

Learn these 3 patterns and you can write all 12

Pattern	Used in	Lines to remember

Read input → act	Exp 2, 3, 4, 5, 12	`x = digitalRead(pin) / analogRead(A0)`
		`Serial.println(x)`
		`digitalWrite(led, ...)`

Ultrasonic	Exp 9, 11	`HIGH on Trig, 10 µs, LOW` `distance = pulseIn(echo, HIGH) * 0.034 / 2`

Wi-Fi + cloud	Exp 1, 6, 7, 8, 11	`WiFi.begin("ssid","pass");`
		`while (WiFi.status() != WL_CONNECTED) delay(500);`
		`then: http.begin → GET/PUT → print code`

Magic numbers

`0.488`	LM35 on Uno: 5 V × 100 ÷ 1024 → °C
`0.322`	LM35 on NodeMCU: 3.3 V × 100 ÷ 1023 → °C
`0.034 / 2`	Ultrasonic: speed of sound in cm/µs, halved for the round trip
`9600 / 115200`	Serial Monitor baud: 9600 for Uno, 115200 for NodeMCU/ESP32
`200`	HTTP success code (ThingSpeak, Firebase)
`15 s`	Minimum gap between ThingSpeak updates (we use 20 s)

All pin connections at a glance

Exp	Board	Pins
1	ESP32	LED → GPIO 2 (on-board)
2	Uno	Button → 5V + pin 4 · 10 kΩ pin 4–GND · LED → pin 13
3	Uno	LDR → 5V + A0 · 10 kΩ A0–GND · LED → 13
4	Uno	LM35: 5V · A0 · GND (left → right)
5	Uno	Relay IN → 8 · Button → pin 2 · Motor via COM/NO
6	NodeMCU	none (USB only)
6b	Uno + ESP-01	ESP TX → 2 · ESP RX ← 3 (divider) · 3.3 V external
7	NodeMCU	LM35: VIN · A0 · G
8	NodeMCU	Soil sensor: 3V3 · G · AO → A0
9	Uno	Trig → 9 · Echo → 10 · LED/buzzer → 13
10	Uno + L298N	IN1 → 8 · IN2 → 9 · common GND
11	NodeMCU	VIN · G · Trig → D5 · Echo → D6 (1k/2k divider)
12	Uno	DHT11: 5V · out → 7 · GND

1. MQTT — control LED from the cloud

Board: ESP32 Dev Module

PIN CONNECTIONS

Component pin	→ Board pin	Note
On-board LED	**GPIO 2**	Built in — nothing to wire
External LED long leg (+) — optional	**GPIO 2 (D2)**	through 220 Ω resistor
External LED short leg (−) — optional	**GND**	

> **Remember:** callback() reacts to messages; loop() just reconnects and calls client.loop(). Send **1** = ON, **0** = OFF.

PROGRAM

```
#include <WiFi.h>
#include <PubSubClient.h>

WiFiClient espClient;
PubSubClient client(espClient);

void callback(char* topic, byte* msg, unsigned int len) {
  if (msg[0] == '1') digitalWrite(2, HIGH);   // "1" = LED ON
  if (msg[0] == '0') digitalWrite(2, LOW);    // "0" = LED OFF
  Serial.println((char)msg[0]);
}

void setup() {
  Serial.begin(115200);
  pinMode(2, OUTPUT);
  WiFi.begin("Your_SSID", "Your_PASSWORD");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("WiFi connected");
  client.setServer("broker.hivemq.com", 1883);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {
    client.connect("esp32_usn123");           // any unique name
    client.subscribe("iot/led");
    Serial.println("MQTT connected");
  }
  client.loop();
}
```

EXPECTED OUTPUT

```
WiFi connected
MQTT connected
1          <- after publishing 1 to iot/led  (LED turns ON)
0          <- after publishing 0 to iot/led  (LED turns OFF)
```

To send the message: HiveMQ websocket client in the browser (broker.hivemq.com, port 8884) → Publish, topic `iot/led`, message `1` or `0`. Library: PubSubClient.

2. LED with push button

Board: Arduino Uno

PIN CONNECTIONS

Component pin	→ Board pin	Note
Push button leg A	**5V**	
Push button leg B	**Pin 4**	
10 kΩ resistor	**Pin 4 ↔ GND**	pull-down: one end at pin 4, other at GND
LED long leg (+)	**Pin 13**	through 220 Ω resistor
LED short leg (−)	**GND**	

Remember: The LED *copies* the button: one line, `digitalWrite(13, digitalRead(4))`.

PROGRAM

```
void setup() {
  pinMode(4, INPUT);      // button
  pinMode(13, OUTPUT);    // LED
}

void loop() {
  digitalWrite(13, digitalRead(4));   // LED copies the button
}
```

EXPECTED OUTPUT

```
No Serial output — observe the LED:
Button pressed   ->  LED ON
Button released  ->  LED OFF
```

3. Light detection with LDR

Board: Arduino Uno

PIN CONNECTIONS

Component pin	→ Board pin	Note
LDR leg 1	**5V**	LDR has no polarity
LDR leg 2	**A0**	
10 kΩ resistor	**A0 ↔ GND**	one end at A0, other at GND
LED	**Pin 13**	on-board LED works — no wiring needed

Remember: Read A0, print it, LED ON if below 400 (dark). Bright = high number.

PROGRAM

```
void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
}

void loop() {
  int light = analogRead(A0);
  Serial.println(light);
  if (light < 400) digitalWrite(13, HIGH);   // dark -> LED ON
  else digitalWrite(13, LOW);                // bright -> LED OFF
  delay(1000);
}
```

EXPECTED OUTPUT

```
812      <- room light   (LED OFF)
795
231      <- hand over LDR (LED ON)
47
```

4. LM35 temperature sensor

Board: Arduino Uno

PIN CONNECTIONS

Component pin	→ Board pin	Note
LM35 left pin (+Vs)	**5V**	hold flat side facing you, legs down
LM35 middle pin (Vout)	**A0**	
LM35 right pin (GND)	**GND**	gets hot if reversed — unplug!

> **Remember:** Reading × **0.488** = °C. (10 mV per °C; 5 V/1024 steps.)

PROGRAM

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  float temp = analogRead(A0) * 0.488;   // 500 / 1024 = 0.488
  Serial.println(temp);
  delay(1000);
}
```

EXPECTED OUTPUT

```
28.30
28.79
28.30
```

5. Relay — motor ON with push button

Board: Arduino Uno

PIN CONNECTIONS

Component pin	→ Board pin	Note
Relay module VCC	**5V**	
Relay module GND	**GND**	
Relay module IN	**Pin 8**	
Push button leg A	**5V**	
Push button leg B	**Pin 2**	
10 kΩ resistor	**Pin 2 ↔ GND**	pull-down
Battery (+)	**Relay COM**	motor side — NOT the Arduino
Relay NO	**Motor wire 1**	NO = normally open
Motor wire 2	**Battery (−)**	

> **Remember:** Same as Exp 2 but pin 8 drives a relay. If it runs backwards (ON when released) the relay is active-LOW: write `!btn`.

PROGRAM

```
void setup() {
  Serial.begin(9600);
  pinMode(2, INPUT);      // button
  pinMode(8, OUTPUT);     // relay IN
}

void loop() {
  int btn = digitalRead(2);
  digitalWrite(8, btn);   // pressed -> relay ON -> motor ON
  Serial.println(btn ? "Motor ON" : "Motor OFF");
  delay(500);
}
```

EXPECTED OUTPUT

```
Motor OFF
Motor OFF
Motor ON     <- while button held (relay clicks)
Motor ON
Motor OFF
```

6. HTTP send & receive (NodeMCU)

Board: NodeMCU 1.0 (ESP-12E Module)

PIN CONNECTIONS

Component pin	→ Board pin	Note
—	—	Nothing to wire. Just plug the NodeMCU in by USB.

> **Remember:** Data goes **in the URL** (field1=45) → that's the *send*; `getString()` is the *receive*. Reply 0 = rejected (too fast or wrong key).

PROGRAM

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

void setup() {
  Serial.begin(115200);
  WiFi.begin("Your_SSID", "Your_PASSWORD");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("WiFi connected");
}

void loop() {
  WiFiClient client;
  HTTPClient http;
  http.begin(client, "http://api.thingspeak.com/update?api_key=YOUR_KEY&field1=45");
  int code = http.GET();                  // SEND
  Serial.println(code);                   // 200 = OK
  Serial.println(http.getString());       // RECEIVE (entry number)
  http.end();
  delay(20000);
}
```

EXPECTED OUTPUT

```
WiFi connected
200          <- HTTP code: request accepted
1            <- server's reply: entry number (counts up: 1, 2, 3...)
200
2
```

Replace YOUR_KEY with the ThingSpeak **Write API Key**.

6b. HTTP with Uno + ESP-01 (only if the lab gives you this)

Board: Arduino Uno

PIN CONNECTIONS

Component pin	→ Board pin	Note
ESP-01 VCC	**External 3.3 V**	NOT Arduino 3.3V (too weak)
ESP-01 EN / CH_PD	**External 3.3 V**	
ESP-01 GND	**GND**	join Arduino GND and supply GND
ESP-01 TX	**Pin 2**	
ESP-01 RX	**Pin 3 via divider**	Pin 3 → 1 kΩ → ESP RX → 2 kΩ → GND

Remember: 4 AT commands in order: **CWMODE → CWJAP → CIPSTART → CIPSEND**, then send the request. loop() just passes text both ways.

PROGRAM

```
#include <SoftwareSerial.h>
SoftwareSerial esp(2, 3);   // RX, TX

void setup() {
  Serial.begin(9600);
  esp.begin(9600);
  esp.println("AT+CWMODE=1");
  delay(2000);
  esp.println("AT+CWJAP=\"Your_SSID\",\"Your_PASSWORD\"");
  delay(8000);
  esp.println("AT+CIPSTART=\"TCP\",\"api.thingspeak.com\",80");
  delay(3000);
  String req = "GET /update?api_key=YOUR_KEY&field1=45 HTTP/1.1\r\n";
  req += "Host: api.thingspeak.com\r\n\r\n";
  esp.print("AT+CIPSEND=");
  esp.println(req.length());
  delay(2000);
  esp.print(req);
}

void loop() {
  if (esp.available()) Serial.write(esp.read());      // show ESP replies
  if (Serial.available()) esp.write(Serial.read());   // type AT commands yourself
}
```

EXPECTED OUTPUT

```
OK
WIFI CONNECTED
WIFI GOT IP
OK
CONNECT
OK
>
SEND OK
+IPD,...:HTTP/1.1 200 OK ...
1
CLOSED
```

Some lines may be missing in the output because replies arrive during the delays — that's fine; check the ThingSpeak channel. If nothing replies, the ESP is at 115200 baud: type `AT+UART_DEF=9600,8,1,0,0` once with esp.begin(115200).

7. Room temperature to ThingSpeak

Board: NodeMCU 1.0 (ESP-12E Module)

PIN CONNECTIONS

Component pin	→ Board pin	Note
LM35 left pin (+Vs)	**VIN**	5 V from USB; LM35 needs ≥ 4 V
LM35 middle pin (Vout)	**A0**	
LM35 right pin (GND)	**G (GND)**	

> **Remember:** Exp 4 + Wi-Fi + **one** line: `ThingSpeak.writeField(channel, field, value, key)`. NodeMCU factor is **0.322**.

PROGRAM

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>

WiFiClient client;

void setup() {
  Serial.begin(115200);
  WiFi.begin("Your_SSID", "Your_PASSWORD");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("WiFi connected");
  ThingSpeak.begin(client);
}

void loop() {
  float temp = analogRead(A0) * 0.322;   // 330 / 1023 = 0.322
  Serial.println(temp);
  ThingSpeak.writeField(1234567, 1, temp, "YOUR_KEY");   // channel, field, value, key
  delay(20000);
}
```

EXPECTED OUTPUT

```
WiFi connected
28.66
28.98          <- one reading every 20 s; same values appear on the ThingSpeak chart
```

Library: ThingSpeak (MathWorks). Replace 1234567 with your channel number.

8. Soil moisture to Firebase

Board: NodeMCU 1.0 (ESP-12E Module)

PIN CONNECTIONS

Component pin	→ Board pin	Note
Sensor module VCC	**3V3**	
Sensor module GND	**G (GND)**	
Sensor module AO	**A0**	AO = analog out (not DO)
Probe (fork) 2 wires	**Sensor module**	either way round

Remember: Read A0 → `http.PUT(String(value))` to `<database-url>/moisture.json`. Dry = high, wet = low.

PROGRAM

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecure.h>

void setup() {
  Serial.begin(115200);
  WiFi.begin("Your_SSID", "Your_PASSWORD");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("WiFi connected");
}

void loop() {
  int moisture = analogRead(A0);
  Serial.println(moisture);

  WiFiClientSecure client;
  client.setInsecure();
  HTTPClient http;
  http.begin(client, "https://your-project-default-rtdb.firebaseio.com/moisture.json");
  Serial.println(http.PUT(String(moisture)));   // 200 = saved in Firebase
  http.end();
  delay(5000);
}
```

EXPECTED OUTPUT

```
WiFi connected
1024      <- sensor in air (dry)
200       <- saved to Firebase
412       <- sensor in wet soil
200
```

Firebase: create Realtime Database in **test mode**, paste its URL. In the console the value updates under `moisture`. Code 401 = rules not in test mode.

9. Intrusion detection (ultrasonic)

Board: Arduino Uno

PIN CONNECTIONS

Component pin	→ Board pin	Note
HC-SR04 VCC	**5V**	
HC-SR04 Trig	**Pin 9**	
HC-SR04 Echo	**Pin 10**	
HC-SR04 GND	**GND**	
LED long leg / buzzer (+)	**Pin 13**	LED through 220 Ω
LED short leg / buzzer (−)	**GND**	

Remember: Trig HIGH 10 µs → `pulseIn(echo) × 0.034 / 2` = cm → if < 20, intruder.

PROGRAM

```
void setup() {
  Serial.begin(9600);
  pinMode(9, OUTPUT);    // Trig
  pinMode(10, INPUT);    // Echo
  pinMode(13, OUTPUT);   // LED / buzzer
}

void loop() {
  digitalWrite(9, HIGH);
  delayMicroseconds(10);
  digitalWrite(9, LOW);
  int distance = pulseIn(10, HIGH) * 0.034 / 2;

  if (distance > 0 && distance < 20) {
    Serial.println("Intruder detected!");
    digitalWrite(13, HIGH);
  } else {
    Serial.println("Area clear");
    digitalWrite(13, LOW);
  }
  delay(500);
}
```

EXPECTED OUTPUT

```
Area clear
Area clear
Intruder detected!     <- hand within 20 cm (LED ON)
Intruder detected!
Area clear
```

10. DC motor direction (L298N)

Board: Arduino Uno

PIN CONNECTIONS

Component pin	→ Board pin	Note
L298N IN1	**Pin 8**	
L298N IN2	**Pin 9**	
L298N ENA	**keep jumper ON**	motor always enabled, full speed
L298N OUT1	**Motor wire 1**	
L298N OUT2	**Motor wire 2**	
L298N 12V terminal	**Battery (+)**	9–12 V
L298N GND terminal	**Battery (−) AND Arduino GND**	common ground — must have both
L298N 5V terminal	**leave empty**	

Remember: 10 = clockwise, 01 = anticlockwise, 00 = stop (IN1, IN2).

PROGRAM

```
void setup() {
  Serial.begin(9600);
  pinMode(8, OUTPUT);    // IN1
  pinMode(9, OUTPUT);    // IN2
}

void loop() {
  digitalWrite(8, HIGH); digitalWrite(9, LOW);
  Serial.println("Clockwise");
  delay(2000);

  digitalWrite(8, LOW);  digitalWrite(9, LOW);
  Serial.println("Stop");
  delay(1000);

  digitalWrite(8, LOW);  digitalWrite(9, HIGH);
  Serial.println("Anticlockwise");
  delay(2000);

  digitalWrite(8, LOW);  digitalWrite(9, LOW);
  Serial.println("Stop");
  delay(1000);
}
```

EXPECTED OUTPUT

```
Clockwise
Stop
Anticlockwise
Stop
(repeats; motor turns 2 s each way with 1 s pause)
```

11. Distance to Google cloud (Firebase)

Board: NodeMCU 1.0 (ESP-12E Module)

PIN CONNECTIONS

Component pin	→ Board pin	Note
HC-SR04 VCC	**VIN**	5 V — sensor won't work on 3V3
HC-SR04 GND	**G (GND)**	
HC-SR04 Trig	**D5**	
HC-SR04 Echo	**1 kΩ → D6**	Echo → 1 kΩ → D6
2 kΩ resistor	**D6 ↔ GND**	drops 5 V echo to 3.3 V

> **Remember:** Exp 9 (with D5/D6) + Exp 8's Firebase lines, path `/distance.json`.

PROGRAM

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecure.h>

void setup() {
  Serial.begin(115200);
  pinMode(D5, OUTPUT);   // Trig
  pinMode(D6, INPUT);    // Echo
  WiFi.begin("Your_SSID", "Your_PASSWORD");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("WiFi connected");
}

void loop() {
  digitalWrite(D5, HIGH);
  delayMicroseconds(10);
  digitalWrite(D5, LOW);
  int distance = pulseIn(D6, HIGH) * 0.034 / 2;
  Serial.println(distance);

  WiFiClientSecure client;
  client.setInsecure();
  HTTPClient http;
  http.begin(client, "https://your-project-default-rtdb.firebaseio.com/distance.json");
  Serial.println(http.PUT(String(distance)));   // 200 = saved
  http.end();
  delay(5000);
}
```

EXPECTED OUTPUT

```
WiFi connected
23       <- distance in cm
200      <- saved to Firebase
9
200
```

Google Cloud IoT Core (named in the syllabus) was shut down in 2023, so Firebase — also Google Cloud — is used.

12. Simple sensor app — DHT11

Board: Arduino Uno

PIN CONNECTIONS

Component pin	→ Board pin	Note
DHT11 + (VCC)	**5V**	3-pin module
DHT11 out (S / DATA)	**Pin 7**	
DHT11 − (GND)	**GND**	bare 4-pin sensor: add 10 kΩ from DATA to 5V

Remember: `DHT dht(7, DHT11)` → `dht.begin()` → readTemperature() / readHumidity(). Prints **nan** = wiring problem.

PROGRAM

```
#include <DHT.h>
DHT dht(7, DHT11);       // data pin, sensor type

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  Serial.print("Temp: ");
  Serial.print(dht.readTemperature());
  Serial.print("  Humidity: ");
  Serial.println(dht.readHumidity());
  delay(2000);
}
```

EXPECTED OUTPUT

```
Temp: 28.00  Humidity: 61.00
Temp: 28.00  Humidity: 62.00
```

Library: DHT sensor library (Adafruit) + Adafruit Unified Sensor.